

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192421166
Name	DHILIBAN M

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I DHILIBAN M/192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Q1. HDFS-Based Storage Solution for a Media Platform

Constraints: Availability > 99.9% | Replication factor ≤ 3 | Storage must be cost-efficient at scale

Constraints Identified & Prioritized

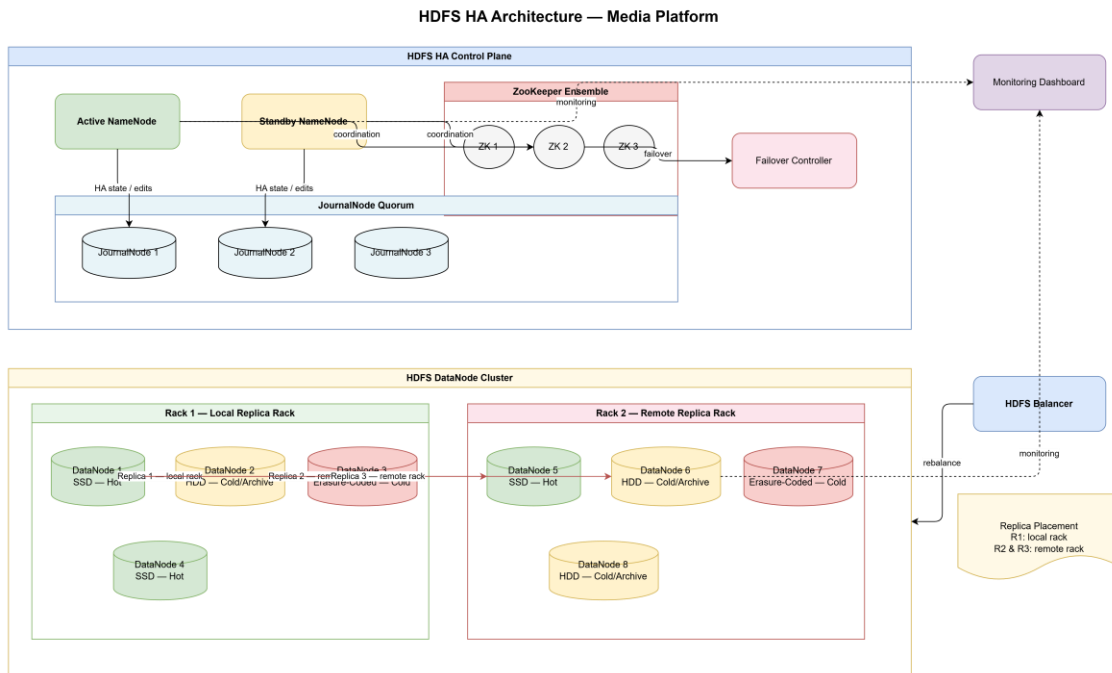
1. **Availability (primary):** 99.9% uptime requires HA NameNode + fault-tolerant DataNode layout — cannot depend on a single point of failure.
2. **Replication ≤ 3 (hard limit):** Restricts redundancy options; must be compensated by rack awareness and lifecycle policies rather than higher replication.
3. **Cost efficiency (secondary, ongoing):** Media files (video/image) are large and grow continuously — storage tiering is essential, since replication cannot be increased to buy safety margin.

Design

- **NameNode HA:** Active-Standby NameNode pair using **Quorum Journal Manager (QJM)** with 3–5 JournalNodes, automatic failover via **ZooKeeper + ZKFailoverController (ZKFC)**. This removes the single point of failure without touching replication factor.
- **Replication = 3, Rack-Aware Placement:** Default HDFS rack-awareness policy — 1st replica on local node/rack, 2nd on a different rack, 3rd on the same remote rack (different node). This gives resilience to both node and rack failure at RF=3, satisfying availability without exceeding the constraint.
- **Storage Tiering for Cost Efficiency:**
 - **Hot tier:** Recently uploaded/actively streamed media on SSD-backed DataNodes (HDFS Heterogeneous Storage — ARCHIVE, DISK, SSD policies).
 - **Cold/Archive tier:** Older, infrequently accessed media auto-moved to ARCHIVE storage policy (dense HDDs) or offloaded to erasure-coded storage.
 - **Erasure Coding (EC) for cold data:** For infrequently accessed large files, use HDFS EC (e.g., RS-6-3) instead of 3x replication — cuts storage overhead from 200% to ~50% while retaining fault tolerance, effectively working around the RF ≤ 3 ceiling for cost.
- **Small-file handling:** Media thumbnails/metadata batched into **HAR (Hadoop Archive)** or stored via **HBase**, avoiding NameNode memory bloat.
- **Monitoring:** HDFS balancer scheduled periodically to prevent DataNode skew; Ambari/Cloudera Manager alerts tied to an SLA dashboard tracking the 99.9% target.

Justification

- HA NameNode + rack-aware RF=3 is the standard proven pattern for >99.9% HDFS availability without exceeding replication limits.
- EC and tiering directly answer the "replication capped, but cost must stay efficient" tension — they provide durability through parity/erasure math instead of full copies for the cold data majority (media platforms are typically 80/20 hot/cold).
- Rack awareness ensures the RF=3 constraint still tolerates a full rack outage, which a naive RF=3 placement would not guarantee.



Q2. MapReduce Job Design for a 20-Minute Log Processing Batch Window

Constraints: Fixed 20-minute SLA | Limited cluster nodes

Constraints Identified & Prioritized

- Time SLA (hard):** Job must complete end-to-end within 20 minutes — dictates split sizing, parallelism, and avoiding long-tail (straggler) tasks.
- Limited nodes (hard, resource ceiling):** Cannot simply scale out — must optimize task count and data locality to make the most of available map/reduce slots.

Job Design

- Input splitting:** Set `mapreduce.input.fileinputformat.split.maxsize` so split size $\approx$ available map slots $\times$ node count, ensuring the number of map tasks matches cluster parallel capacity (avoid over-splitting into thousands of tiny tasks that add scheduling overhead, and avoid under-splitting that leaves nodes idle).
- Combiner usage:** Add a **Combiner** identical/similar to the Reducer logic (e.g., partial log-count aggregation) to cut shuffle volume drastically before network transfer — critical when node count is limited and network I/O is the bottleneck.
- Data locality:** Rely on HDFS block placement so map tasks run on nodes already holding the data (rack-local/node-local preferred over data-local fallback), minimizing network shuffle time inside the 20-minute window.
- Reducer count tuning:** Set reducer count close to (but not exceeding) available reduce slots (`mapreduce.job.reduces  $\approx$  number of nodes  $\times$  reduce slots per node`) to run all reducers in a single wave rather than multiple sequential waves.

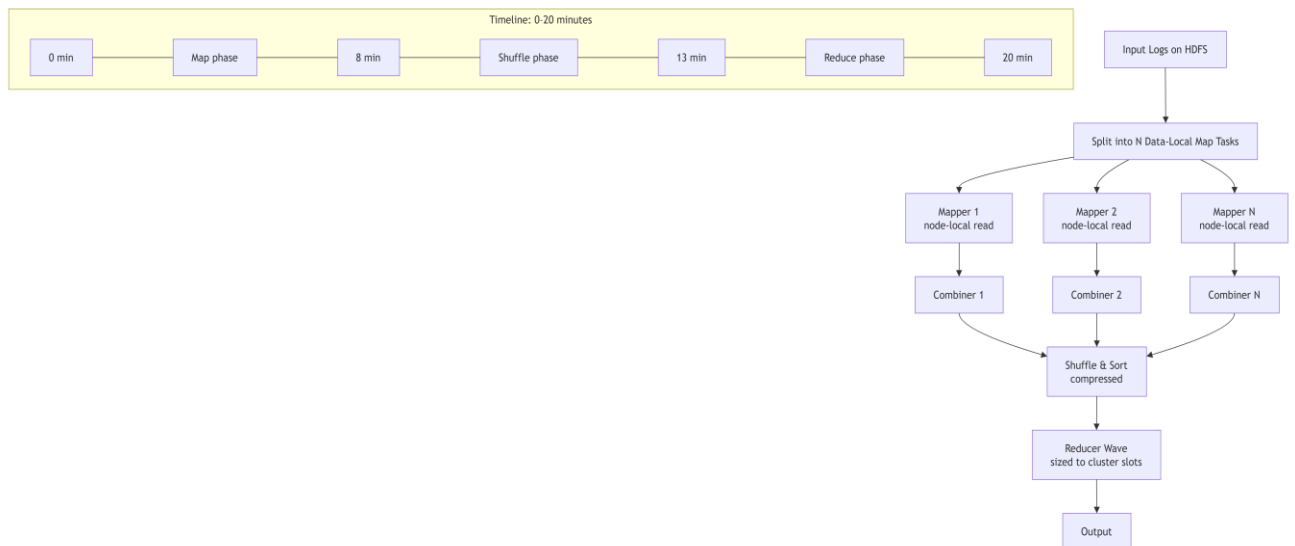
- **Speculative execution:** Enable `mapreduce.map.speculative / reduce.speculative = true`, so slow/straggling tasks on a busy node are duplicated elsewhere and the fastest result wins — protects the SLA against a single slow node.
- **Compression:** Enable intermediate compression (Snappy/LZ4) for map output to shrink shuffle/sort time, again valuable under a limited-node network ceiling.
- **Partitioning strategy:** Use a custom Partitioner if log keys (e.g., by service/date) are skewed, to prevent a single reducer from becoming the long pole.

Task Distribution Explanation

- Map phase: splits distributed 1:1 with data blocks already local to nodes → parallel map wave completes without network wait.
- Shuffle phase: combiner-reduced output + compression → smaller transfer, so shuffle finishes well inside the time budget even with few nodes.
- Reduce phase: single wave sized to available slots → no queuing delay for a second wave of reducers.

Justification

- Combiners and locality-aware splitting directly target the two levers under scarce nodes: reducing data movement and maximizing per-node parallel work.
- Speculative execution and skew-aware partitioning protect against the classic failure mode of batch SLA breaches — a single straggler holding up the whole job.



Q3. YARN Scheduler Policy for ETL, Reporting, and ML Workloads

Constraints: Fairness across job types | ETL must get priority | Must recover gracefully from node failure

Constraints Identified & Prioritized

1. **Fairness (baseline):** All three workload types (ETL, reporting, ML) need guaranteed minimum resources — no starvation.

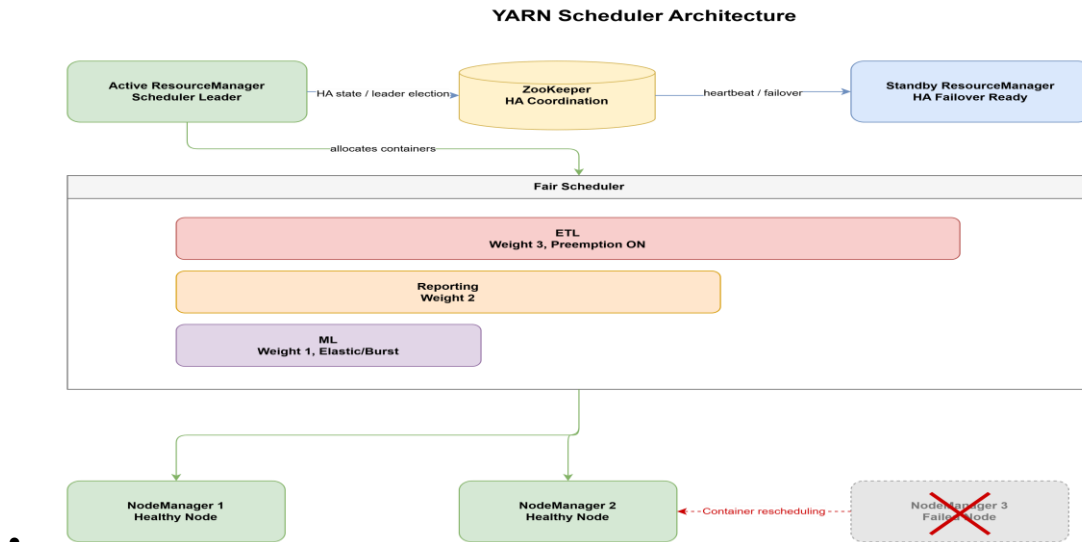
2. **ETL priority (override on top of fairness):** Business-critical ETL pipelines must preempt or get resources ahead of reporting/ML when contention occurs.
3. **Node-failure recovery (resilience):** Scheduler and running applications must recover without manual intervention or SLA breach.

Scheduler Policy Design

- **Scheduler choice: YARN Fair Scheduler with weighted queues** (Capacity Scheduler is a valid alternative, but Fair Scheduler's preemption model maps more naturally to "fair by default, priority on demand").
- **Queue structure:**
 - root.etl — **weight 3**, minimum resource guarantee (e.g., 50% of cluster), preemption **enabled** so ETL can reclaim resources from lower-priority queues within a configured timeout.
 - root.reporting — weight 2, minimum guarantee ~25%, preemption disabled (reporting jobs are shorter, tolerate queuing).
 - root.ml — weight 1, minimum guarantee ~25%, run in **elastic mode** (can burst into idle capacity from other queues when ETL/reporting are quiet, e.g., overnight).
- **Fairness mechanism:** Within each queue, Fair Scheduler allocates resources equally among running apps (fairSharePreemptionTimeout), preventing one large ML job from starving other ML jobs.
- **Priority mechanism:** yarn.scheduler.fair.preemption=true with minSharePreemptionTimeout set low for the ETL queue — if ETL doesn't get its minimum share within X seconds, YARN preempts containers from reporting/ML queues to satisfy ETL first.
- **Node failure recovery:**
 - **ResourceManager HA** (Active/Standby RM with ZooKeeper-based leader election and RMStateStore in ZooKeeper) ensures scheduler state itself survives a master failure.
 - **NodeManager failure:** ResourceManager detects missed heartbeats, marks the node lost, and reschedules its containers on healthy nodes; **MapReduce ApplicationMaster retry** (yarn.resourcemanager.am.max-attempts) restarts affected job attempts automatically.
 - **Work-preserving recovery:** Enable yarn.nodemanager.recovery.enabled=true so NodeManager restarts (not just RM) don't kill in-flight containers unnecessarily.

Justification

- Weighted Fair Scheduler queues give a mathematically fair floor to every workload while minSharePreemptionTimeout gives ETL an enforceable priority lane — satisfying both constraints simultaneously rather than trading one for the other.
- RM HA + NodeManager recovery + AM retries form YARN's standard three-layer resilience model, ensuring failure recovery doesn't depend on any single component.



Q4. Ingestion Architecture for SAN, NAS, and Operational Databases

Constraints: Reliable backup | Minimal duplication | Multiple heterogeneous sources (SAN, NAS, RDBMS)

Constraints Identified & Prioritized

1. **Minimal duplication (primary technical challenge):** Three different source types with overlapping data domains risk redundant copies if ingested naively.
2. **Backup reliability (non-negotiable):** Ingested data must be recoverable — ingestion pipeline itself must not become a single point of data loss.
3. **Heterogeneity of sources:** SAN/NAS are file/block-level storage; operational DBs are structured — each needs a different ingestion mechanism, but must land in one governed platform.

Constrained Architecture

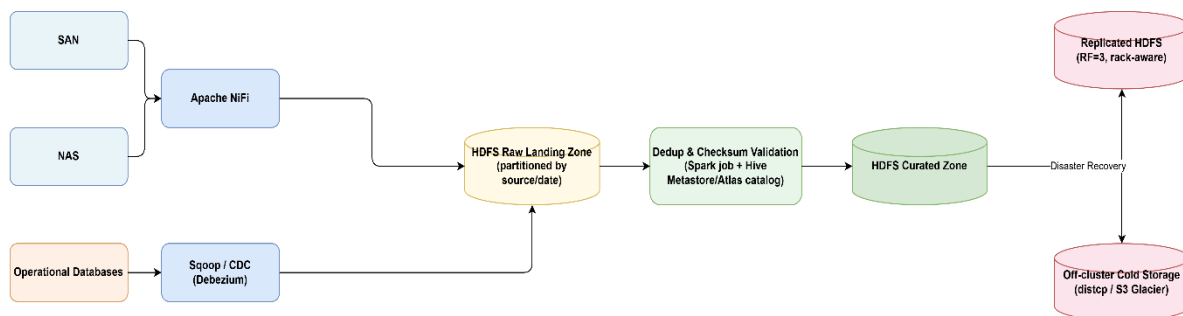
- **NAS ingestion:** Use **Apache NiFi** with GetFile/ListFile + FetchFile processors (or HDFS FileSystem connector) to pull files from NAS shares (typically media/log/document files) directly into a **landing zone in HDFS**, preserving file checksums (MD5/CRC) at ingest for later dedup verification.
- **SAN ingestion:** SAN is block-level; front it with a filesystem-mounted gateway server, then use the same NiFi file-based flow, or a scheduled distcp-style batch pull if SAN exposes an NFS/CIFS mount — avoid duplicating SAN's own backup role by ingesting only designated export volumes, not full LUN mirrors.
- **Operational DB ingestion:** Use **Apache Sqoop** (batch, full/incremental import via --incremental append/lastmodified) or **NiFi CDC / Kafka Connect + Debezium** for near-real-time change-data-capture — CDC avoids re-pulling the entire table each run, directly minimizing duplication.
- **Central landing + dedup layer:**
 - All sources land in a single **HDFS raw zone**, partitioned by source/date.

- A lightweight **hash-based dedup job** (Spark/MapReduce) compares checksums against a metadata catalog (e.g., **Apache Atlas** or **Hive metastore** tags) before promoting data from raw → curated zone, ensuring the same file/record isn't ingested twice from overlapping sources (e.g., a report that exists both on NAS and as a DB export).
- **Backup reliability:**
 - HDFS itself is replicated (RF=3) across nodes/racks as the first backup layer.
 - **Secondary backup:** Scheduled distcp snapshots of the curated zone to a **separate cluster or cloud cold storage (e.g., S3/Glacier-class)** for disaster recovery, independent of the primary cluster's failure domain.
 - **HDFS Snapshots** enabled on ingestion directories for point-in-time rollback in case a bad ingestion run corrupts data.

Justification

- Using source-appropriate ingestion tools (NiFi for files, Sqoop/CDC for databases) avoids forcing one tool to badly handle all source types, which is a common cause of duplicate or incomplete pulls.
- CDC over full re-import is the direct lever for "minimal duplication" from operational databases — it's the single highest-impact design choice for that constraint.
- The two-tier backup (in-cluster replication + off-cluster distcp/snapshot) satisfies reliability without depending on ingestion pipeline uptime alone.

Ingestion Architecture and Disaster Recovery



Q5. End-to-End Hadoop Ecosystem Solution (Secure Movement, HA, NiFi/ETL Integration)

Constraints: Secure data movement | High availability | Integration with Apache NiFi / ETL pipelines

Constraints Identified & Prioritized

1. **Security in transit and at rest (hard, compliance-driven):** Every hop — ingestion, storage, processing, consumption — must be encrypted and access-controlled.
2. **High availability (hard, operational):** No single component (NameNode, ResourceManager, NiFi) can be a single point of failure.

3. **NiFi/ETL integration (functional requirement):** The whole pipeline must be orchestrated, not just a set of standalone tools.

End-to-End Design

1. Ingestion Layer — Apache NiFi (Clustered)

- NiFi run as a **NiFi Cluster** (3+ nodes) coordinated via **ZooKeeper**, so ingestion continues if one NiFi node fails.
- All source connections use **TLS/SSL** (https, sftp, encrypted JDBC connections); NiFi's **Site-to-Site** protocol between clusters also runs over TLS.
- NiFi processors tagged with data provenance for audit trail (built-in NiFi Provenance repository).

2. Storage Layer — HDFS (HA)

- Active/Standby NameNode with QJM + ZooKeeper (as in Q1), RF=3, rack-aware.
- **Encryption at rest:** HDFS Transparent Data Encryption (TDE) using **HDFS Encryption Zones** backed by **Apache Ranger KMS** (Key Management Server) — data is encrypted per-zone, keys never touch disk in plaintext.
- **Data in transit inside cluster:** dfs.encrypt.data.transfer=true (RPC + block data encryption between clients/DataNodes).

3. Access Control & Authentication

- **Kerberos** for cluster-wide strong authentication (NiFi, HDFS, YARN, Hive all Kerberized).
- **Apache Ranger** for centralized, fine-grained authorization (row/column-level policies on Hive, HDFS directory ACLs, NiFi resource policies) — single policy plane across the ecosystem.

4. Processing Layer — YARN (HA) + ETL

- ResourceManager HA (as in Q3), Fair Scheduler queues for ETL/analytics.
- ETL transformation via **NiFi → Hive/Spark on YARN**: NiFi handles ingestion + light routing/transformation, heavier transformation (joins, aggregations) delegated to **Spark jobs on YARN** for scale, orchestrated by **Apache Oozie** or **Apache Airflow** for scheduling and dependency management.

5. Serving/Consumption Layer

- Curated data exposed via **Hive** (secured by Ranger, encrypted connections) for reporting, or pushed to downstream systems via NiFi's secured Site-to-Site output ports.

How the Design Satisfies Each Constraint

Constraint	Mechanism
Secure data movement	TLS everywhere (NiFi in/out), Kerberos authentication, HDFS transfer encryption, Ranger-enforced authorization

Constraint	Mechanism
High availability	NameNode HA (QJM+ZK), ResourceManager HA, clustered NiFi (ZK-coordinated), RF=3 rack-aware storage
NiFi/ETL integration	NiFi as ingestion/orchestration entry point, Spark/Hive for heavy ETL, Oozie/Airflow for pipeline scheduling — all Ranger/Kerberos-secured

Justification

- Kerberos + Ranger + TLS is the industry-standard "defense in depth" security stack for Hadoop (authentication, authorization, and encryption addressed as three separate concerns, not conflated).
- HA is applied consistently at *every* stateful component (NameNode, RM, NiFi) rather than just one — a partial HA design (e.g., HA NameNode but single NiFi node) would leave a hidden single point of failure.
- Splitting NiFi (ingestion/light-ETL/orchestration entry) from Spark (heavy transformation) reflects how NiFi is architecturally intended to integrate with Hadoop, rather than overloading NiFi with big-data-scale processing it isn't built for.

